

Day 7 – Webhook Fundamentals

(Synchronous)

Goals: Understand webhook mechanics — payload signing with HMAC, HTTP delivery, and the downsides of doing it synchronously.

Extend the management command with a synchronous webhook sender:

```
games/webhooks.py:
```

```
import hmac
import hashlib
import json
import requests

def send_expiry_webhook(publisher, game_key_str, game_title, expired_at):
    payload = {
        "event": "game_key.expired",
        "game_key": game_key_str,
        "game_title": game_title,
        "expired_at": expired_at.isoformat(),
    }

    secret = publisher.webhook_secret.encode()
    body = json.dumps(payload, sort_keys=True).encode()
    signature = hmac.new(secret, body, hashlib.sha256).hexdigest()

    headers = {
        "Content-Type": "application/json",
        "X-Signature": f"sha256={signature}",
    }

    try:
        response = requests.post(publisher.webhook_url, json=payload, headers=headers)
        response.raise_for_status()
    except Exception as e:
        print(f"[Webhook] Delivery failed for publisher {publisher.id}:
```

Updated check_expired_keys.py using sync delivery:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey
from games.webhooks import send_expiry_webhook

class Command(BaseCommand):
    help = 'Mark expired keys and notify publishers synchronously.'

    def handle(self, *args, **options):
```

```

expired_keys = GameKey.objects.select_related('game__publisher'
    expires_at__lte=timezone.now(),
    status='active'
)
count = expired_keys.update(status='expired')

for key in GameKey.objects.filter(status='expired').select_rela
    send_expiry_webhook(key.game.publisher, key.key_string, key

self.stdout.write(f'Expired {count} keys (sync webhooks sent).')

```

Discussion points:

- Synchronous HTTP calls block the management command thread
- A slow or unavailable publisher endpoint delays all subsequent notifications
- No retry logic — a failed delivery is silently lost
- → Solution: move webhook delivery to an async task queue (Day 8)

Instructor Teaching Plan

Time Breakdown (90 min)

Segment	Duration	Activity
What is a webhook?	20 min	Contrast webhooks with polling. Real-world examples: GitHub fires a webhook when a PR is merged; Stripe fires one when a payment succeeds. Draw the pub/sub flow: publisher registers a URL → our system fires HTTP POST when something happens → publisher handles it.
HMAC signing	20 min	Why sign? Because anyone can POST to the publisher's webhook URL. HMAC lets the publisher verify the payload came from us and wasn't tampered with. Walk through the algorithm: <code>secret + payload → SHA256 digest</code> . Show <code>sort_keys=True</code> and why it matters for reproducibility.
Write <code>webhooks.py</code>	20 min	Code the synchronous sender together. Test with webhook.site — a free endpoint that logs every request. Students should see the payload arrive.
Update management command	10 min	Integrate sync sender. Run manually. Observe blocking behavior (if webhook.site is slow, the command waits).
Discussion: sync limitations	10 min	Go through the 3 bullet points at the bottom of Day 7. Collect student ideas on how to fix each. Preview: Day 8 solves all three with Celery.
Q&A + checkpoint	10 min	

Key Concepts to Introduce

- **Webhook** = **HTTP callback** — instead of the subscriber polling "has anything changed?", the publisher pushes "something changed!" to a pre-registered URL. Push > pull for latency and efficiency.

- **HMAC-SHA256** — Hash-based Message Authentication Code. Both parties share a secret. The sender computes `HMAC(secret, payload)` and sends it as a header. The receiver recomputes it and compares — if they match, the payload is authentic and untampered.
- **`sort_keys=True` in `json.dumps()`** — JSON object key order is not guaranteed. If the receiver serializes the same data differently (different key order), the signature won't match. `sort_keys=True` makes the serialization deterministic.
- **`hmac.compare_digest()`** — constant-time comparison, not `==`. Prevents timing attacks where an attacker could determine the correct signature byte by byte by measuring response time.
- **`requests.post(..., timeout=5)`** — always set a timeout on outbound HTTP calls. Without it, your process can hang forever if the publisher's server is slow or down.
- **`raise_for_status()`** — raises `HTTPError` for 4xx/5xx responses. Without it, a 500 from the publisher looks like success.

⚠️ Common Mistakes to Address

- **`hmac.new()` vs `hmac.HMAC()`** — Python's `hmac` module uses `hmac.new()` (older API) or `hmac.HMAC()` (Python 3.10+). Both work; be consistent. Common bug: mixing them.
- **Not encoding strings to bytes** — `hmac.new(secret, body, ...)` requires bytes. `secret.encode()` and `body` (already bytes from `.encode()`) — forgetting either raises `TypeError`.
- **Not handling exceptions** — if `requests.post()` raises a `ConnectionError` (publisher server is down), the whole management command crashes without processing the other keys. Wrap in `try/except`.
- **No timeout on `requests.post()`** — if the publisher's server never responds, the management command hangs indefinitely. `timeout=5` is a reasonable default.
- **Reusing payload dict** — if you modify `payload` after signing (e.g., to add extra fields), the signature no longer matches. Serialize and sign at the same point.

? Check for Understanding

"A publisher claims they received a webhook from us but the payload was tampered with. How does HMAC protect against this?"

Answer: HMAC uses a shared secret to generate a cryptographic signature of the request body. If any character in the payload is altered during transit, the receiver's computed signature will not match the `X-Signature` header, revealing that the payload was modified and should be rejected.

"Why do we pass `sort_keys=True` to `json.dumps()`? What could go wrong without it?"

Answer: JSON keys are unordered. If we don't sort keys, the serialization format might change (e.g. dictionary keys serialized in a different order), resulting in a different byte sequence. This would produce a different HMAC signature, causing verification to fail even if the content remains identical.

"What happens to the management command if the publisher's server is down and we're making synchronous HTTP calls?"

Answer: The management command thread will block, waiting for the HTTP request to timeout (e.g. 5 seconds per request). If many keys expired for that publisher, or multiple publishers are down, the script will take a very long time to complete and could cause other pending updates to stall.

"What's the difference between a 4xx and 5xx response from the publisher's webhook endpoint? Should we retry both?"

Answer: A 4xx response represents a client error (e.g., 400 Bad Request or 404 Not Found), suggesting our payload is invalid or the endpoint is misconfigured; retrying will likely fail again, so we shouldn't retry. A 5xx response represents a server error (e.g., 500 Internal Server Error or 503 Service Unavailable), suggesting temporary publisher outage; we should retry these as the server might recover.

✓ Day Checkpoint

Student uses webhook.site as the publisher's `webhook_url`. After running `python manage.py check_expired_keys`, the `webhook.site` log shows the POST request with the correct JSON body and X-Signature header. The student can manually verify the HMAC in a Python shell.